🚨

Design an Earthquake notification system

功能性需求（Functional Requirements）

1. 使用者可以設定要接收通知的條件（距離 distance、震度 magnitude）。
2. 當地震發生時，系統能通知符合條件的使用者。

非功能性需求（Non-Functional Requirements）

1. 系統需以低延遲（sub-seconds）將警報送達所有目標使用者。
2. 我們應該盡量避免重複推送通知（duplicate notifications）。

API 設計（API design）

- 使用者設定地震通知條件

```
POST /alerts/configuration -> Success/Error
{
  magnitude,
  distance,
  ...
}
```

- 使用者定期回報所在位置

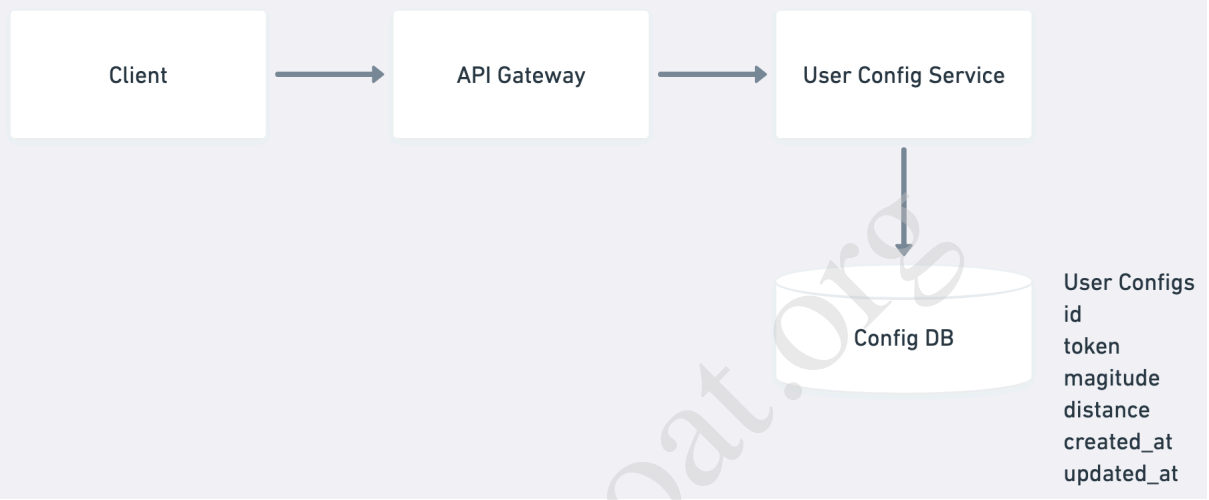
```
POST /alerts/user_location
{
  lat,
  long
```

buildmoat.org

```
}
# 使用者身分 token 會以 JWT 放在 header
```

High-Level Design

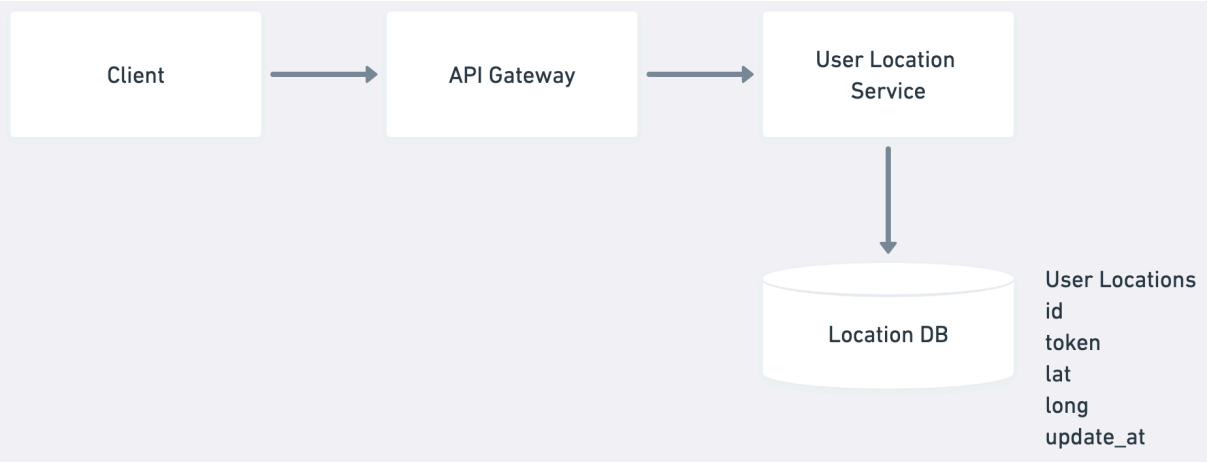
1. User configuration flow（使用者設定流程）



1. 使用者送出包含 alert 設定的 POST request。
2. API Gateway 將請求轉送到後端服務。
3. 服務將使用者設定寫入 DB。
4. 在註冊流程中，系統會先給一組預設設定，或在 DB 中加入 `status` 欄位，將尚未送出設定的使用者標記為 `inactive` 。

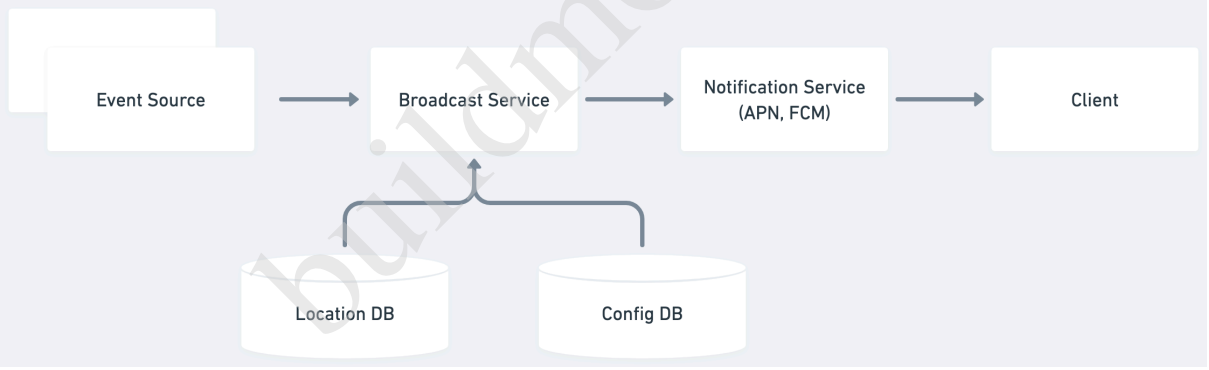
2. User Location Reporting flow（位置回報流程）

buildmoat.org



1. 使用者裝置會定期回傳目前位置。
2. 不需要非常頻繁更新，因為位置精準度要求不高（例如使用者移動超過 100km 通常需要數小時）。
3. 由於位置更新頻率遠高於設定更新，會使用獨立的 service 與 database 儲存 location data。

3. Alert broadcast flow（警報廣播流程）



1. Event source：來自地震中心等外部資料來源。取得資料的方式有很多種，其中一個例子是對警報來源維持 persistent feed，讓我們能夠穩定地接收連續的資料串流。
2. Broadcast service 接收事件後，查詢 location DB 與 config DB，計算哪些使用者需要被通知。
3. 確認目標名單後，將通知送到第三方推播服務（iOS 使用 APNs，Android 使用 FCM）進行 fan-out。

深入探討（Deep Dives）

buildmost.org

1. 我們要如何處理使用者位置更新，並且有效率地找出需要通知的使用者？

1. **Frequency of writes（寫入頻率）**

   假設我們有 2,000 萬使用者（例如台灣），每位使用者每小時回報一次位置：

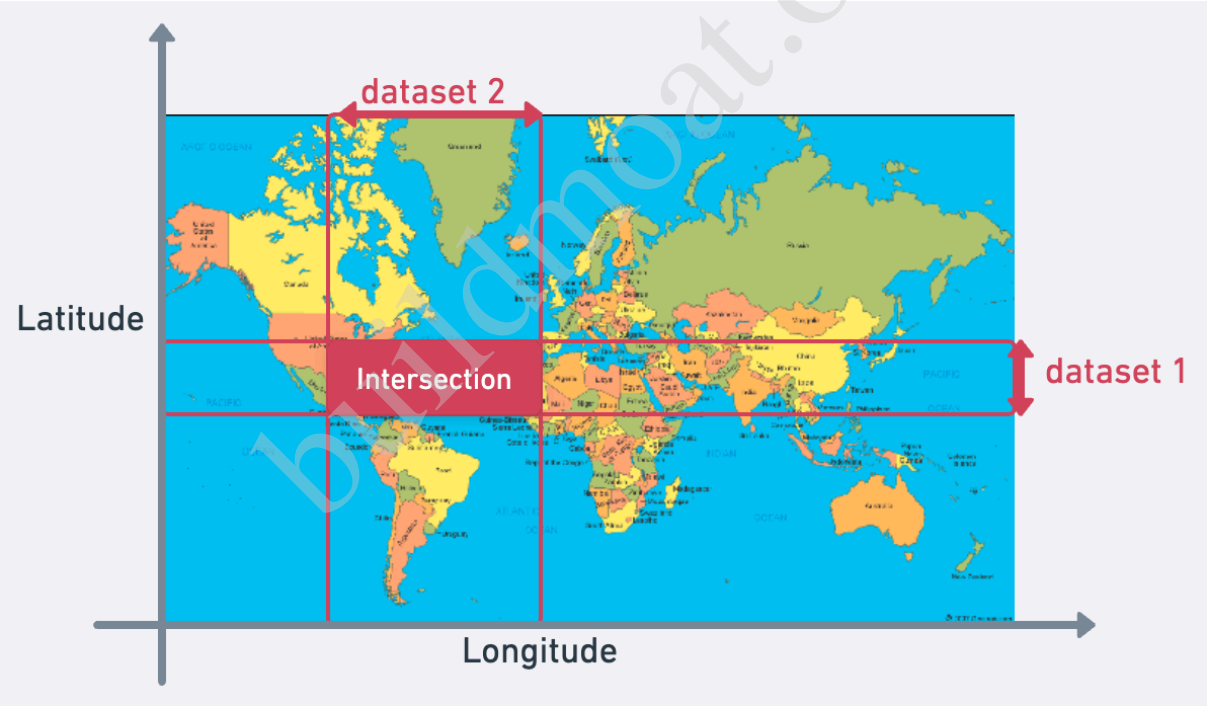
   20M / 3600 ≈ 5.5K writes/s（每秒約 5,500 次寫入）

2. **Query Efficiency（查詢效率）**

   在沒有任何最佳化的情況下，如果用 lat/long 來查詢資料表，就必須對整張表做 full table scan，對每一位使用者計算其位置與地震事件位置的距離。

   這在有數百萬使用者時會極度低效。即使對 lat/long 欄位建立 index，傳統的 B-tree index 也不適合多維度資料（如地理座標），在做 proximity search 時效能仍然很差。

   這基本上是個 non-starter（不可行方案）。



buildmoat.org

Geo Index 選項比較

- **Geohash（字串格網）**
  - 將經緯度用二分法切成長方形網格，再編碼成 Base32 字串。字串前綴相同代表空間位置相近。
  - 優點：簡單、普及、生態成熟。

    - 缺點：格子長寬比不一致、靠近兩極變形嚴重，圓形或多邊形覆蓋時會出現鋸齒邊界。
    - https://geohash.softeng.co/
- **S2（Google 開源）**
    - 將地球投影到一個立方體的 6 個面上，每個面用四分樹（quadtree）切割成 S2 Cells（球面上的近似正方形），每個 Cell 有 64-bit ID。
    - 優點：階層結構規整、RegionCoverer 做多邊形覆蓋很強、工業界大量使用。
    - 缺點：非等面積（高緯度略有變形）、概念與實作稍複雜。
    - https://igorgatis.github.io/ws2/
- **H3（Uber 開源）**
    - 在二十面體上鋪六角形格網（少量五邊形用於封閉曲面），每一層把一個六角形細分成 7 個更小六角形，每個 Cell 是 64-bit 整數 ID。
    - 優點：六角形鄰接均勻、面積更接近等面積，內建 `kRing`、`polyfill`，聚合操作方便。
    - 缺點：存在五邊形例外，與行政邊界貼合度不如 S2 彈性。
    - https://h3geo.org/
- 多邊形覆蓋需求：

  S2 / H3 提供原生 polyfill / RegionCoverer，可用少量且緊密的 cells 覆蓋多邊形；Geohash 以矩形拼接容易產生鋸齒邊界且 cell 數暴增。
- 均勻性與可控性：

  H3 六角形接近等面積；S2 變形可預期、誤差易控制；Geohash 在高緯度面積失真明顯。
- 工程實作面：

  S2 與 H3 都有 64-bit 階層式索引，適合做 cell → devices 的快取、分片與去重，查詢速度快、成本低。

因此，在地震這種「多邊形影響範圍 → 找出受影響使用者」的場景中，選擇 S2/H3 會比 Geohash 更準確、更省資源、也更容易維護。

Options（設計方案）

1. **(Naive)**

   每次 alert 都直接用 raw lat/long 查 OLTP table 並計算距離。

buildmoat.org

- 為何失敗：需要 full / near-full scan；B-tree 不適合 2D geometry；無法 scale 到百萬使用者。

2. **(Better)**

   使用 NoSQL + 自訂 geo key（Geohash / S2 / H3）做持久化：

   將使用者位置以 cell_id（而非 raw lat/long）存入 Cassandra / Dynamo。

   - Pros：高寫入吞吐、可水平擴充、以 cell_id 分片很簡單。
   - Cons：仍需額外結構支援即時推播；多邊形處理需自行實作。

3. **(Best for hot path)**

   S2/H3 cell grid + KV index（per channel）：

   用固定解析度的 cells 表示地球，在高速 KV（Redis / Aerospike / Dynamo + DAX）中維護 cell → device_refs，並保留 device → cells 反向索引以便清理。

   1. **Writes**：

      App 回報的是 **cell ids**（不是 lat/long），設定 TTL（7–30 天），可輕鬆支援 O(100k)/s。

   2. **Lookups（alert time）**：

      對 alert polygon 做 polyfill → 得到 cell 清單 → 依序串流取出 chunked device refs → 去重 → 依 channel enqueue。

   3. Pros：低延遲、行為可預測、操作簡單、對隱私較友善（只存粗粒度 cell）。
   4. Cons：邊界可能有少量 false positives，除非再做精細化裁切。

2. 我們要如何把警報快速送達所有受影響的使用者？

1. **(Naive solution)**

   接收 event source 更新的 server 直接把 notification 推送給所有使用者。

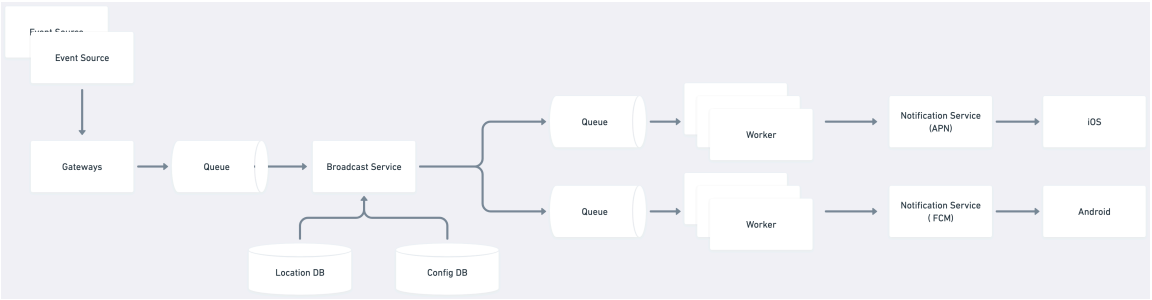
   這種作法顯然有兩個主要問題：

   1. 當受影響的使用者數量很多時，我們無法「快速」通知到所有人（ALL the impacted users fast）。
   2. 系統用來送 notification 的資源，會和從 event source 拉資料所需的資源互相競爭。

      一旦推播發送端出現問題，甚至可能拖慢 event pulling process，進而影響後續的事件通知。

buildmoat.org

2. **(Improved)** 使用 queues 將邏輯解耦（decouple logic）以隔離失敗（isolate failures）：



1. 我們引入以下幾個元件（components）：

   a. **Gateway**：

   維持與 alert sources 的 persistent feeds，負責處理 TLS、heartbeats、reconnect、exponential backoff，以及資料來源端的各種 quirks（rate limits、retry-after、malformed payload quarantine）。

   b. **Broadcast Service (Orchestrator)**：

   負責 geo-targeting，並將事件路由到各個 per-channel queues。

      1. 在 Redis 中維護 `cell → device tokens` 的對應表。
      2. 當 alert 來時：
         - 用 cells 覆蓋 alert polygon
         - 從 cell:devices index 拉出候選裝置
         - 去重（de-dupe）
         - 依 channel 分組成 chunks（例如每組 500–2,000 個 device refs）
         - 每個 chunk 封裝成一個 job 丟進 queue。

   c. **Workers**：

   輕量、快速、stateless 的處理程序，從 per-channel queue 中取出已決定好的 “send job”，實際呼叫 APNs / FCM / SMS vendor 送推播。

      1. 每個 sender worker 取出一個 job（也就是一個 chunk），逐一對 chunk 裡的 token 發送 push（因為 APNs / FCM 不支援真正的 broadcast call）。

   d. **Broadcast queues 與 workers 以 per-channel 為單位**，

buildmoat.org

這樣可以隔離故障，並針對不同通道套用不同的 retry / rate limit policy。

2. 在 **Gateway 與 Broadcast Service** 之間放一層 queue，好處是：
    a. Persistent ingest（Gateway）與 CPU-heavy 的 targeting/rendering（Orchestrator）可以獨立 scaling。
    b. Fail soft：即使 Orchestrator 重啟，Gateway 仍能持續接收事件，不會掉資料。
    c. 可以快速演進 Orchestrator 的邏輯，而不會影響與 event source 之間脆弱的 long-lived connections。
3. 在 **Broadcast Service 與 Workers** 之間再放一層 queue：
    a. Orchestrator 能快速完成 geo-targeting 並丟 job 進 queue；queue 吸收流量尖峰，workers 依照 vendor 安全速率（APNs/FCM/SMS quotas）慢慢消化。
    b. 若 APNs 發生 brownout，只會讓 APNs queue 堆積；FCM / SMS 仍可正常流動，Orchestrator 也不受影響。
    c. 可依 channel / region 獨立擴充 workers，不影響 decisioning；每個 channel 可有不同 shard / partition 數量。
4. 為了極低延遲（fast delivery），queue 可選用 **Redis Stream**：
    - Redis 是 memory-first、協調成本低。
    - 相較之下 Kafka 是 disk-first，設計目標是高吞吐與高持久性的 log，而非極低延遲 fan-out。

3. 我們如何避免重複推播（duplicate notifications）？

對於重複通知，我們需要考慮兩種情況：

1. **Duplicates**：同一個 alert 的同一個 version，絕對不應該被送到同一位使用者兩次。
2. **Out-of-order**：對於同一個事件（incident），較新的更新必須要能覆蓋（supersede）較舊的通知。

為了達成這個目標，我們需要引入兩個新的元件（components）：

1. **Supersession cache**：
    `supersession:{alert_id} -> latest_version`
    - 用來記錄某個 `alert_id` 目前已知的最新版本（latest_version）。

buildmoat.org

2. 一張資料表 `OutBox`，作為每一個 device-level notification 的：
   - idempotency gate（冪等性關卡）
   - latest-status ledger（最新狀態帳本）

```
notification_outbox
  notification_id,
  alert_id,
  version,
  device_id,
  channel,
  status,
  ...
```

說明：

1. `status` 可能為：
   `ENQUEUED | ATTEMPED | VENDOR_ACCEPTED | FAILED | CANCELLED_SUPERSEDED`
2. Terminal states 為 `ATTEMPED` 之後的所有狀態。
3. `notification_id = hash(alert_id | version | device_id)`
4. 每個 `notification_id` 只保留一筆記錄（1 record per notification_id）。

Avoiding duplicates（避免同版本重複送出）

1. **Orchestrator**：
   a. 計算 `notification_id = hash(alert_id | version | device_id)`
   b. 若該 `notification_id` 在 Outbox 中已存在且狀態為 terminal state，則不再 enqueue。
   c. 將 chunk job 放入 per-channel queue，並把狀態更新為 `ENQUEUED`。
2. **Sender Worker**：
   a. 讀取 Outbox 紀錄；若為 terminal state，直接 ack / drop 該 job。
   b. 實際呼叫 APNs / FCM 發送推播，並 upsert 狀態為 `ATTEMPTED`。
   c. 收到 APNs / FCM 回應後，再將狀態 upsert 為 `VENDOR_ACCEPTED`（或對應的 failure code）。

Avoiding out-of-order alerts（處理版本覆蓋 supersession）

buildmoat.org

1. **Orchestrator**：
    a. 更新 supersession cache；若該 alert 的 version 比目前記錄的 latest_version 舊，直接 drop 掉。
    b. 對同一個 `alert_id`，將 Outbox 中所有 `version < latest_version` 且尚未進入 terminal state 的紀錄，批次更新為 `CANCELLED_SUPERSEDED`。
2. **Sender Worker**：
    a. 讀取 supersession cache。

    若該 job 的 version < latest_version，則跳過發送，並在該 `notification_id` 尚未進入 terminal state 的情況下，將自己的 Outbox 狀態更新為 `CANCELLED_SUPERSEDED`。

buildmoat.org